

NAME: A VELMURUGAN

LAB EXPERIMENT : LESSON 4

EXPERIMENT NO: 1

Online Python Compiler and Visualizer

Python3

```
1 def dice(s,d,t):
2     dp=[0]*(t+1)
3     for _ in range(d+1):
4         dp[0]=1
5         for i in range(1,d+1):
6             for j in range(1,t+1):
7                 dp[i][j]=sum(dp[i-1][j-k] for k in range(1,s+1) if j>=k)
8     return dp[d][t]
9
10 s,d,t=map(int,input().split())
11 print(dice(s,d,t))
```

Run Visualize Code

6 2 7

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 9.308 Mb

Sample Input

6 2 7

Your Output

6

EXPERIMENT NO:2

Python3

```
1 def assembly(a,t,e,x):
2     f=[e[0]+a[0][0],e[1]+a[1][0]]
3     for i in range(1,len(a[0])):
4         f=[a[0][i]+min(f[0],f[1]+t[1][i-1]),
5             a[1][i]+min(f[1],f[0]+t[0][i-1])]
6     return min(f[0]+x[0],f[1]+x[1])
7
8 n=int(input())
9 a=[list(map(int,input().split())) for _ in range(2)]
10 t=[list(map(int,input().split())) for _ in range(2)]
11 e=list(map(int,input().split()))
12 x=list(map(int,input().split()))
13 print(assembly(a,t,e,x))
```

Run Visualize Code

7 9 3 4 8 4
8 5 6 4 5 7
2 4
3 2

Output

Time: 0.0300 secs Memory: 11.7 Mb

Sample Input

6
7 9 3 4 8 4
8 5 6 4 5 7
2 4
3 2

EXPERIMENT NO:3

The screenshot shows a Python IDE with a code editor on the left and a console/output area on the right. The code implements a dynamic programming solution for a 3x3 grid problem. The grid is defined as follows:

0	0	4
7	6	5
0	2	3

The code calculates the minimum path sum from the top-left cell (0,0) to the bottom-right cell (2,2). The output of the program is 16.

```
Python3
1 n=int(input())
2 a=[list(map(int,input().split())) for _ in range(3)]
3 t=[list(map(int,input().split())) for _ in range(3)]
4
5 dp=a[0]
6 for i in range(1,n):
7     dp=[a[i][j]+min(dp[k]+t[k][i] for k in range(3))
8         for j in range(3)]
9 print(min(dp))
```

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.236 Mb

Sample Input

```
3
5 9 3
6 8 4
7 6 5
0 2 3
2 0 4
3 4 0
```

Your Output

```
16
```

EXPERIMENT NO:4

The screenshot shows a Python IDE with a code editor on the left and a console/output area on the right. The code uses `itertools.permutations` to generate all permutations of a sequence of numbers. The sequence is defined as follows:

0	10	15	20
10	0	35	25
15	35	0	30
20	25	30	0

The code calculates the minimum sum of a sequence of numbers, where the sequence is defined as follows:

0	10	15	20
10	0	35	25
15	35	0	30
20	25	30	0

The output of the program is 80.

```
Python3
from itertools import permutations

n=int(input())
a=[list(map(int,input().split())) for _ in range(n)]

ans=min(sum(a[p[i]][p[i+1]] for i in range(n-1))+a[p[-1]][p[0]]
        for p in permutations(range(n)))
print(ans)
```

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.36 Mb

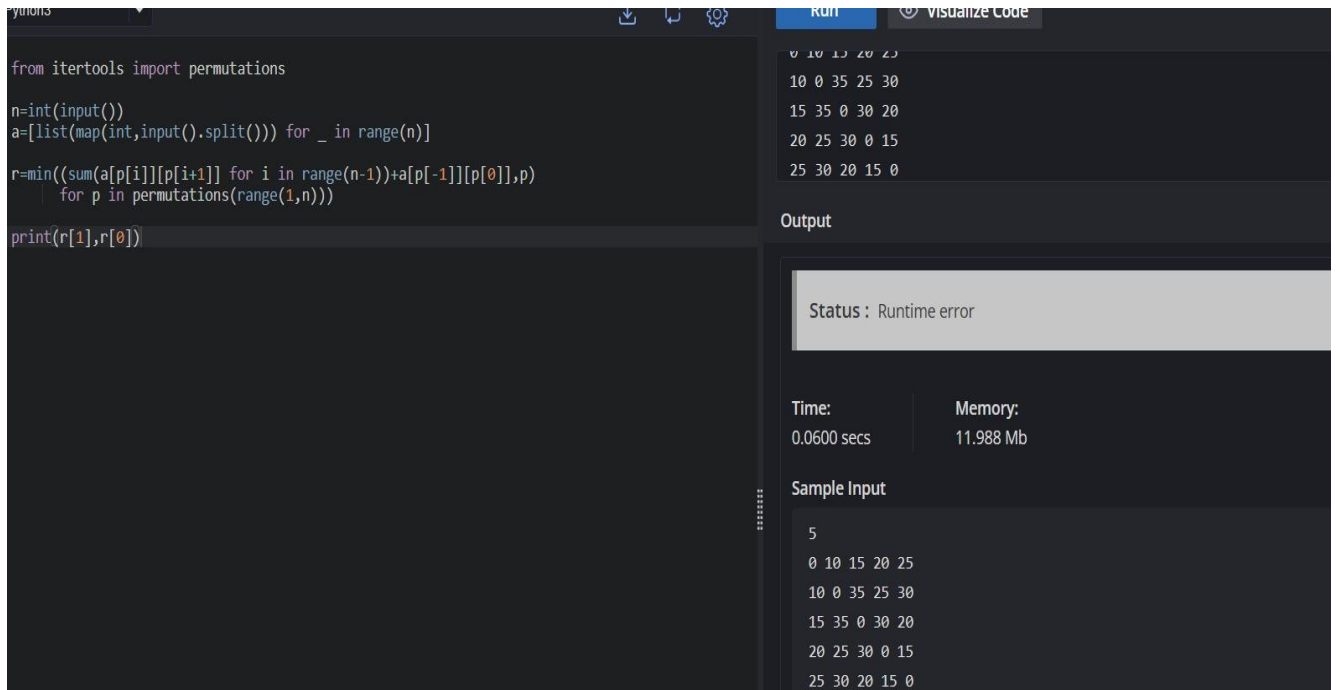
Sample Input

```
4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
```

Your Output

```
80
```

EXPERIMENT NO:5



```
from itertools import permutations

n=int(input())
a=[list(map(int,input().split())) for _ in range(n)]

r=min((sum(a[p[i]][p[i+1]] for i in range(n-1))+a[p[-1]][p[0]],p)
      for p in permutations(range(1,n)))

print(r[1],r[0])
```

Output

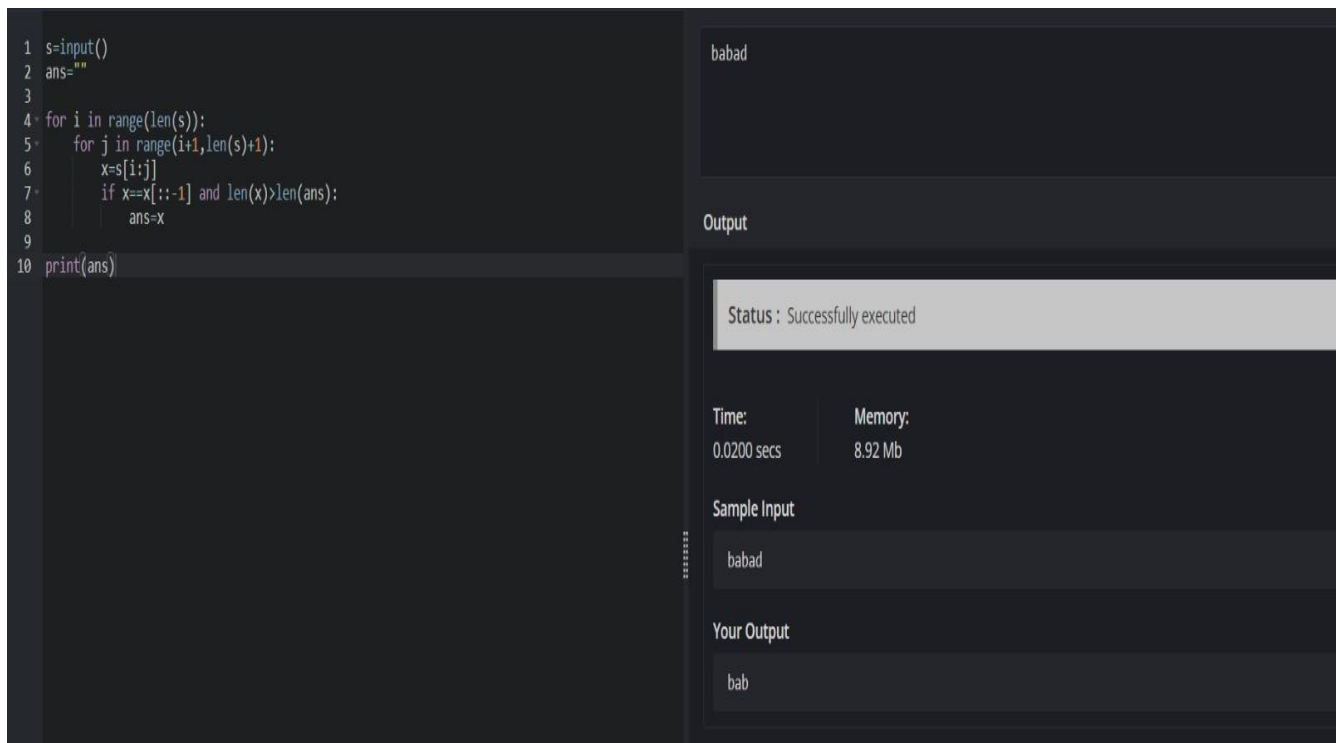
Status : Runtime error

Time: 0.0600 secs Memory: 11.988 Mb

Sample Input

```
5
0 10 15 20 25
10 0 35 25 30
15 35 0 30 20
20 25 30 0 15
25 30 20 15 0
```

EXPERIMENT NO:6



```
1 s=input()
2 ans=""
3
4 for i in range(len(s)):
5     for j in range(i+1,len(s)+1):
6         x=s[i:j]
7         if x==x[::-1] and len(x)>len(ans):
8             ans=x
9
10 print(ans)
```

babad

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.92 Mb

Sample Input

```
babad
```

Your Output

```
bab
```

EXPERIMENT NO:7

```
1 s=input()
2 a=""
3 ans=0
4
5 for c in s:
6     if c in a:
7         a=a[a.index(c)+1:]
8     a+=c
9     ans=max(ans,len(a))
10
11 print(ans)
```

abcbcb

Output

Status : Successfully executed

Time:	Memory:
0.0300 secs	8.648 Mb

Sample Input

abcbcb

Your Output

3

EXPERIMENT NO:8

```
s=input()
d=input().split()

dp=[True]+[False]*len(s)

for i in range(1,len(s)+1):
    dp[i]=any(dp[j] and s[j:i] in d for j in range(i))

print(dp[-1])
```

leetcode
leet code

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	8.928 Mb

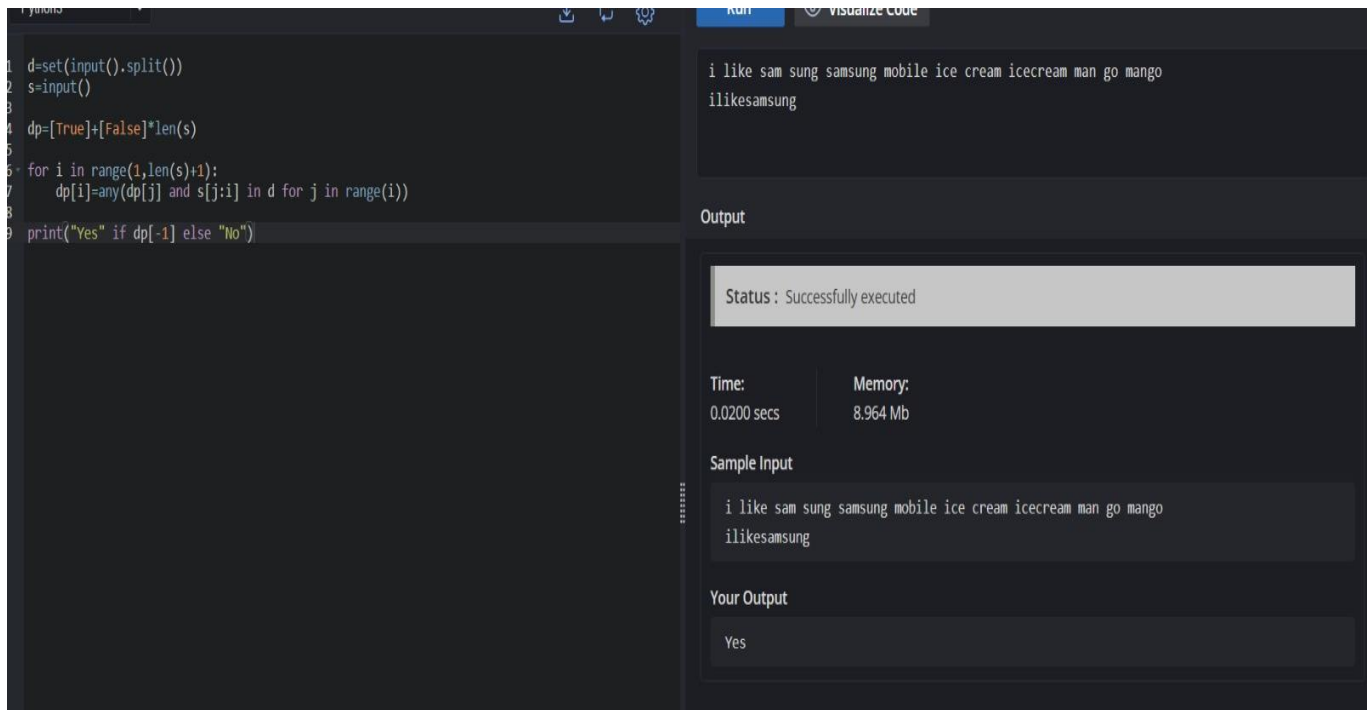
Sample Input

leetcode
leet code

Your Output

True

EXPERIMENT NO:9



```
1 d=set(input().split())
2 s=input()
3
4 dp=[True]+[False]*len(s)
5
6 for i in range(1,len(s)+1):
7     dp[i]=any(dp[j] and s[j:i] in d for j in range(i))
8
9 print("Yes" if dp[-1] else "No")
```

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.964 Mb

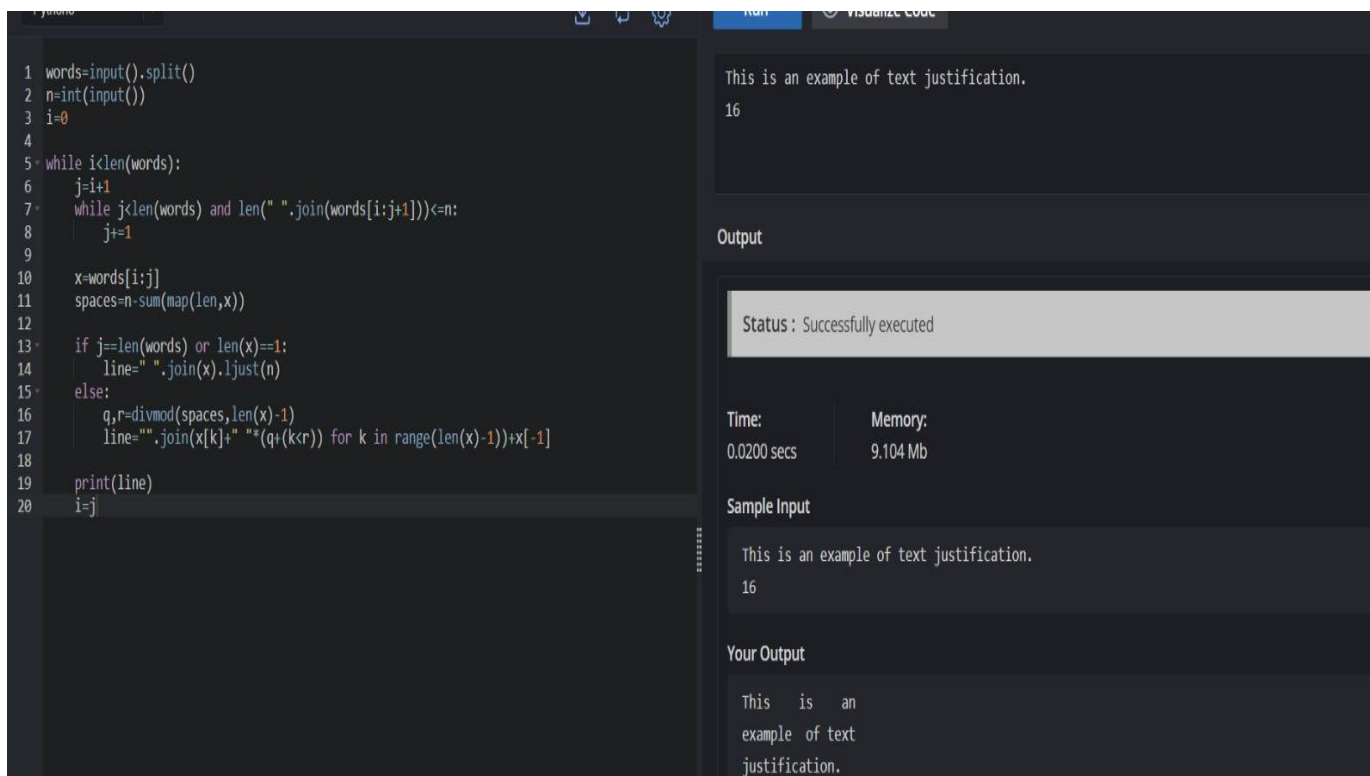
Sample Input

i like sam sung samsung mobile ice cream icecream man go mango
ilikesamsung

Your Output

Yes

EXPERIMENT NO:10



```
1 words=input().split()
2 n=int(input())
3 i=0
4
5 while i<len(words):
6     j=i+1
7     while j<len(words) and len(" ".join(words[i:j+1]))<=n:
8         j+=1
9
10    x=words[i:j]
11    spaces=n-sum(map(len,x))
12
13    if j==len(words) or len(x)==1:
14        line=" ".join(x).ljust(n)
15    else:
16        q,r=divmod(spaces,len(x)-1)
17        line=" ".join(x[k]+" "+(q+(k<r)) for k in range(len(x)-1))+x[-1]
18
19    print(line)
20    i=j
```

Output

This is an example of text justification.
16

Status : Successfully executed

Time: 0.0200 secs Memory: 9.104 Mb

Sample Input

This is an example of text justification.
16

Your Output

This is an
example of text
justification.

EXPERIMENT NO:11

```
1 n=int(input())
2 w=input().split()
3 p,s=input().split()
4
5 for i in range(n-1,-1,-1):
6     if w[i].startswith(p) and w[i].endswith(s):
7         print(i)
8         break
9 else:
10    print(-1)
```

1
apple
a e

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.12 Mb

Sample Input

1
apple
a e

Your Output

0

EXPERIMENT NO:12

```
1 n=int(input())
2 d=[list(map(int,input().split())) for _ in range(n)]
3
4 for k in range(n):
5     for i in range(n):
6         for j in range(n):
7             d[i][j]=min(d[i][j],d[i][k]+d[k][j])
8
9 for r in d:
10    print(*r)
```

4
0 3 999 -4
999 0 4 1
2 999 0 999
999 6 -5 0

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.108 Mb

Sample Input

4
0 3 999 -4
999 0 4 1
2 999 0 999
999 6 -5 0

Your Output

-7 -4 -9 -11

EXPERIMENT NO:14

```
1 n=int(input())
2 e=int(input())
3 d=[[999]*n for _ in range(n)]
4
5 for i in range(n): d[i][i]=0
6
7 for _ in range(e):
8     a,b,w=map(int,input().split())
9     d[a][b]=d[b][a]=w
10
11 t=int(input())
12
13 for k in range(n):
14     for i in range(n):
15         for j in range(n):
16             d[i][j]=min(d[i][j],d[i][k]+d[k][j])
17
18 c=[sum(d[i][j]<=t for j in range(n) if i!=j) for i in range(n)]
19 print(min((c[i],i) for i in range(n))[1]*-1)
```

```
4 5
1 2 1
1 3 4
2 3 1
4
```

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.324 Mb

Sample Input

```
4
4
0 1 3
1 2 1
1 3 4
2 3 1
4
```

Your Output

```
3
```

EXPERIMENT NO:15

```
Python3
1 def obst(f):
2     n=len(f)
3     d=[[0]*n for _ in range(n)]
4
5     for i in range(n):
6         d[i][i]=f[i]
7
8     for l in range(2,n+1):
9         for i in range(n-l+1):
10             j=i+l-1
11             d[i][j]=sum(f[i:j+1])+min(
12                 (d[i][k-1] if k>i else 0)+(d[k+1][j] if k<j else 0)
13                 for k in range(i,j+1))
14
15     return d[0][-1]
16
17 f=list(map(int,input().split()))
18 print(obst(f))
```

```
34 50
```

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.72 Mb

Sample Input

```
34 50
```

Your Output

```
118
```

EXPERIMENT NO:16

```
1 f=list(map(int,input().split()))
2 n=len(f)
3 d=[[0]*n for _ in range(n)]
4
5 for i in range(n):
6     d[i][i]=f[i]
7
8 for l in range(2,n+1):
9     for i in range(n-l+1):
10         j=i+l-1
11         d[i][j]=sum(f[i:j+1])+min(
12             (d[i][k-1] if k>i else 0)+(d[k+1][j] if k<j else 0)
13             for k in range(i,j+1))
14
15 print(d[0][-1])
```

4 2 6 3

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.008 Mb

Sample Input

4 2 6 3

Your Output

26

EXPERIMENT NO:17

```
1 def game(g, m, c, turn, seen):
2     if m == 0:
3         return 1
4     if m == c:
5         return 2
6
7     state=(m,c,turn)
8     if state in seen:
9         return 0
10    seen.add(state)
11
12    moves=g[m] if turn==0 else g[c]
13
14    for x in moves:
15        if turn==1 and x==0:
16            continue
17
18        if turn==0:
19            r=game(g,x,c,1,seen)
20        else:
21            r=game(g,m,x,0,seen)
22
23        if r == (1 if turn==0 else 2):
24            return r
25
26    return 0
27
28 n=int(input())
29 g=[]
30
31 for i in range(n):
32     g.append(list(map(int,input().split())))
33
34 print(game(g,1,2,0,set()))
```

0 4 5
1 4 5
2 3
0 2 3

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 9.18 Mb

Sample Input

6
2 5
3
0 4 5
1 4 5
2 3
0 2 3

Your Output

0

EXPERIMENT NO:18

The screenshot shows a Python IDE with a dark theme. The code on the left implements a heap sort algorithm. The input is read from the user, converted to a list of integers, and then sorted using the heap sort function. The output is printed as a space-separated string.

```
import heapq

n = int(input())
arr = list(map(int, input().split()))

def heap_sort(arr):
    n = len(arr)
    for i in range(n//2 - 1, -1, -1):
        heapq.heapify(arr, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapq.heapify(arr, 0)
    return arr

sorted_arr = heap_sort(arr)
print(' '.join(map(str, sorted_arr)))
```

The output panel on the right shows the execution results. The input is 3, followed by the list [0,1],[1,2],[0,2], and the sorted output 0.5 0.5 0.2.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.624 Mb

Sample Input

3
[[0,1],[1,2],[0,2]]
0.5 0.5 0.2
0 2

Your Output

0.25000

EXPERIMENT NO:19

The screenshot shows a Python IDE with a dark theme. The code on the left implements a dynamic programming solution for finding the maximum sum of a subsequence. The input is read from the user, and the maximum sum is calculated using a dynamic programming array. The output is printed as a space-separated string.

```
Python3

1 m,n=map(int,input().split())
2 dp=[1]*n
3
4 for i in range(m-1):
5     for j in range(1,n):
6         dp[j]=dp[j-1]
7
8 print(dp[-1])
```

The output panel on the right shows the execution results. The input is 3 7, and the output is 28.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.916 Mb

Sample Input

3 7

Your Output

28

EXPERIMENT NO:20

```
Python3
1 a=list(map(int,input().split()))
2 d={}
3 ans=0
4
5 for x in a:
6     ans+=d.get(x,0)
7     d[x]=d.get(x,0)+1
8
9 print(ans)
```

1 2 3 1 1 3

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.212 Mb

Sample Input

1 2 3 1 1 3

Your Output

4

EXPERIMENT NO:21

```
Python3
1 n=int(input())
2 e=int(input())
3 d=[[999]*n for _ in range(n)]
4
5 for i in range(n):d[i][i]=0
6
7 for _ in range(e):
8     a,b,w=map(int,input().split())
9     d[a][b]=d[b][a]=w
10
11 t=int(input())
12
13 for k in range(n):
14     for i in range(n):
15         for j in range(n):
16             d[i][j]=min(d[i][j],d[i][k]+d[k][j])
17
18 c=[sum(d[i][j]<=t for j in range(n) if i!=j) for i in range(n)]
19 print(min((c[i],-i) for i in range(n))[1]*-1)
```

0 1 3
1 2 1
1 3 4
2 3 1
4

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 9.292 Mb

Sample Input

4
4
0 1 3
1 2 1
1 3 4
2 3 1
4

Your Output

3

EXPERIMENT NO:22

```
1 import heapq
2
3 n,m=map(int,input().split())
4 g=[[] for _ in range(n+1)]
5
6 for _ in range(m):
7     a,b,w=map(int,input().split())
8     g[a].append((b,w))
9
10 k=int(input())
11 d=[999]*(n+1)
12 d[k]=0
13 q=[(0,k)]
14
15 while q:
16     x,u=heapq.heappop(q)
17     if x>d[u]:continue
18
19     for v,w in g[u]:
20         if x+w<d[v]:
21             d[v]=x+w
22             heapq.heappush(q,(d[v],v))
23
24 print(max(d[1:]) if max(d[1:])<999 else -1)
```

```
4 3
2 1 1
2 3 1
3 4 1
2
```

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.152 Mb

Sample Input

```
4 3
2 1 1
2 3 1
3 4 1
2
```

Your Output

```
2
```

```
1 imp
2
3 n,m
4 g=
5
```

Snipping Tool

Screenshot copied
Automatically saved